

Python: exercises on classes

This notebook was generated in **solutions** mode.

Exercise 1: Simple Person Class

Exercise Description

Create a class called `Person` with attributes `name` and `age`. Add a method called `introduce` that prints `"Hello, my name is [name] and I am [age] years old."`.

Example:

```
p = Person("Alice", 30)
p.introduce()
# Output: Hello, my name is Alice and I am 30 years old.
```

After creating the class, create a `Person` object with name "Alice" and age 30, and store it in a variable called `result`.

Your solution

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

# Create a Person object
result = Person("Alice", 30)
```

Test your solution

```
# Test the Person class
assert hasattr(result, 'name'), "Person should have a name attribute"
assert hasattr(result, 'age'), "Person should have an age attribute"
assert result.name == "Alice", "Person name should be Alice"
assert result.age == 30, "Person age should be 30"
assert hasattr(result, 'introduce'), "Person should have an introduce method"
```

Test your solution

```
# Test creating different Person objects
p2 = Person("Bob", 25)
assert p2.name == "Bob"
assert p2.age == 25

# Test that objects are independent
```

```
assert result.name != p2.name
assert result.age != p2.age
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Dog Class with Default Values

Exercise Description

Create a class called `Dog` with attributes `name` and `breed`. If no breed is provided, default it to `"Unknown"`. Add a method `bark` that prints `"Woof! I am a [breed]!"`.

Example:

```
d1 = Dog("Max", "Golden Retriever")
d1.bark()
# Output: Woof! I am a Golden Retriever!

d2 = Dog("Buddy")
```

```
d2.bark()  
# Output: Woof! I am an Unknown!
```

After creating the class, create two `Dog` objects: one with name "Max" and breed "Golden Retriever", and another with just name "Buddy". Store them in variables called `result1` and `result2`.

Your solution

```
class Dog:  
    def __init__(self, name, breed="Unknown"):  
        self.name = name  
        self.breed = breed  
  
    def bark(self):  
        print(f"Woof! I am a {self.breed}!")  
  
# Create Dog objects  
result1 = Dog("Max", "Golden Retriever")  
result2 = Dog("Buddy")
```

Test your solution

```
# Test the Dog class with specified breed  
assert hasattr(result1, 'name'), "Dog should have a name attribute"  
assert hasattr(result1, 'breed'), "Dog should have a breed attribute"  
assert result1.name == "Max", "First dog name should be Max"  
assert result1.breed == "Golden Retriever", "First dog breed should be Golden Retriever"  
assert hasattr(result1, 'bark'), "Dog should have a bark method"
```

Test your solution

```
# Test the Dog class with default breed
assert result2.name == "Buddy", "Second dog name should be Buddy"
assert result2.breed == "Unknown", "Second dog breed should default to Unknown"

# Test that objects are independent
assert result1.name != result2.name
assert result1.breed != result2.breed
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

Exercise 3: Rectangle Class with Methods

Exercise Description

Create a class `Rectangle` with attributes `length` and `width`. Add a method `area` that returns the area of the rectangle and a method `perimeter` that returns the perimeter.

Example:

```
r = Rectangle(5, 3)
print(r.area())      # Output: 15
print(r.perimeter()) # Output: 16
```

After creating the class, create a `Rectangle` object with length 5 and width 3, and store it in a variable called `result`.

Your solution

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

# Create a Rectangle object
result = Rectangle(5, 3)
```

Test your solution

```
# Test the Rectangle class
assert hasattr(result, 'length'), "Rectangle should have a length attribute"
assert hasattr(result, 'width'), "Rectangle should have a width attribute"
assert result.length == 5, "Rectangle length should be 5"
```

```
assert result.width == 3, "Rectangle width should be 3"
assert hasattr(result, 'area'), "Rectangle should have an area method"
assert hasattr(result, 'perimeter'), "Rectangle should have a perimeter method"
```

Test your solution

```
# Test the methods
assert result.area() == 15, "Area should be 15 (5 * 3)"
assert result.perimeter() == 16, "Perimeter should be 16 (2 * (5 + 3))"

# Test with different rectangle
r2 = Rectangle(4, 6)
assert r2.area() == 24, "Area should be 24 (4 * 6)"
assert r2.perimeter() == 20, "Perimeter should be 20 (2 * (4 + 6))"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Bank Account Class

Exercise Description

Create a class `BankAccount` with attributes `balance` (initialized to 0). Add methods `deposit` to add to the balance and `withdraw` to subtract from the balance.

Example:

```
account = BankAccount()  
account.deposit(100)  
account.withdraw(40)  
print(account.balance)  # Output: 60
```

After creating the class, create a `BankAccount` object, deposit 100, withdraw 40, and store the account in a variable called `result`.

Your solution


```
class BankAccount:
    def __init__(self):
        self.balance = 0

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount

# Create and use BankAccount object
result = BankAccount()
result.deposit(100)
result.withdraw(40)
```

Test your solution

```
# Test the BankAccount class
assert hasattr(result, 'balance'), "BankAccount should have a balance attribute"
assert hasattr(result, 'deposit'), "BankAccount should have a deposit method"
assert hasattr(result, 'withdraw'), "BankAccount should have a withdraw method"
assert result.balance == 60, "Balance should be 60 after deposit 100 and withdraw 40"
```

Test your solution

```
# Test with new account
account2 = BankAccount()
assert account2.balance == 0, "New account should start with balance 0"

account2.deposit(50)
```

```
assert account2.balance == 50, "Balance should be 50 after deposit"

account2.withdraw(20)
assert account2.balance == 30, "Balance should be 30 after withdraw"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Private Bank Account Class

Exercise Description

Modify `BankAccount` to have a private `balance` attribute (e.g., `_balance`). Ensure users can only view or change the balance through `deposit`, `withdraw`, and `get_balance` methods.

Example:

```
account = BankAccount()
account.deposit(100)
```

```
account.withdraw(40)
print(account.get_balance())  # Output: 60
```

After creating the class, create a `BankAccount` object, deposit 100, withdraw 40, and store the account in a variable called `result`.

Your solution

```
class BankAccount:
    def __init__(self):
        self._balance = 0

    def deposit(self, amount):
        self._balance += amount

    def withdraw(self, amount):
        self._balance -= amount

    def get_balance(self):
        return self._balance

# Create and use BankAccount object
result = BankAccount()
result.deposit(100)
result.withdraw(40)
```

Test your solution

```
# Test the BankAccount class with private balance
assert hasattr(result, '_balance'), "BankAccount should have a _balance private attri
```

```
assert hasattr(result, 'deposit'), "BankAccount should have a deposit method"
assert hasattr(result, 'withdraw'), "BankAccount should have a withdraw method"
assert hasattr(result, 'get_balance'), "BankAccount should have a get_balance method"
assert result.get_balance() == 60, "Balance should be 60 after deposit 100 and withdraw 40"
```

Test your solution

```
# Test with new account
account2 = BankAccount()
assert account2.get_balance() == 0, "New account should start with balance 0"

account2.deposit(75)
assert account2.get_balance() == 75, "Balance should be 75 after deposit"

account2.withdraw(25)
assert account2.get_balance() == 50, "Balance should be 50 after withdraw"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Vehicle Inheritance

Exercise Description

Create a `Vehicle` class with an attribute `speed` and a method `move`. Then, create a subclass `Car` that adds an attribute `fuel`.

Example:

```
v = Vehicle(60)
c = Car(100, "Full")
```

After creating the classes, create a `Vehicle` object with speed 60 and a `Car` object with speed 100 and fuel "Full". Store them in variables called `result1` and `result2`.

Your solution

```
class Vehicle:
    def __init__(self, speed):
        self.speed = speed

    def move(self):
        print(f"Moving at {self.speed} km/h")

class Car(Vehicle):
    def __init__(self, speed, fuel):
        super().__init__(speed)
        self.fuel = fuel

# Create objects
```

```
result1 = Vehicle(60)
result2 = Car(100, "Full")
```

Test your solution

```
# Test the Vehicle class
assert hasattr(result1, 'speed'), "Vehicle should have a speed attribute"
assert hasattr(result1, 'move'), "Vehicle should have a move method"
assert result1.speed == 60, "Vehicle speed should be 60"

# Test the Car class
assert hasattr(result2, 'speed'), "Car should have a speed attribute"
assert hasattr(result2, 'fuel'), "Car should have a fuel attribute"
assert hasattr(result2, 'move'), "Car should inherit move method"
assert result2.speed == 100, "Car speed should be 100"
assert result2.fuel == "Full", "Car fuel should be Full"
```

Test your solution

```
# Test inheritance
assert isinstance(result2, Car), "result2 should be an instance of Car"
assert isinstance(result2, Vehicle), "result2 should also be an instance of Vehicle"
assert not isinstance(result1, Car), "result1 should not be an instance of Car"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Person String Representation

Exercise Description

Modify `Person` to add `__str__` to return a readable representation.

Example:

```
p = Person("Bob", 25)
print(p)  # Output: Bob, 25 years old
```

After creating the class, create a `Person` object with name "Bob" and age 25, and store it in a variable called `result`.

Your solution

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, {self.age} years old"
```

```
# Create a Person object
result = Person("Bob", 25)
```

Test your solution

```
# Test the Person class
assert hasattr(result, 'name'), "Person should have a name attribute"
assert hasattr(result, 'age'), "Person should have an age attribute"
assert result.name == "Bob", "Person name should be Bob"
assert result.age == 25, "Person age should be 25"
assert hasattr(result, '__str__'), "Person should have a __str__ method"
```

Test your solution

```
# Test string representation
assert str(result) == "Bob, 25 years old", "String representation should be 'Bob, 25"

# Test with different person
p2 = Person("Alice", 30)
assert str(p2) == "Alice, 30 years old", "String representation should be 'Alice, 30"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```


Exercise 8: Calculator Static Method

Exercise Description

Create a class `Calculator` with a static method `add`.

Example:

```
print(Calculator.add(5, 3))  # Output: 8
```

After creating the class, call `Calculator.add(5, 3)` and store the result in a variable called `result`.

Your solution

```
class Calculator:
    @staticmethod
    def add(x, y):
        return x + y

# Call the static method
result = Calculator.add(5, 3)
```

Test your solution

```
# Test the Calculator class
assert hasattr(Calculator, 'add'), "Calculator should have an add method"
assert result == 8, "Calculator.add(5, 3) should return 8"

# Test that it's a static method (can be called without instance)
assert Calculator.add(10, 20) == 30, "Calculator.add(10, 20) should return 30"
```

Test your solution

```
# Test with different values
assert Calculator.add(0, 0) == 0, "Calculator.add(0, 0) should return 0"
assert Calculator.add(-5, 5) == 0, "Calculator.add(-5, 5) should return 0"
assert Calculator.add(100, 200) == 300, "Calculator.add(100, 200) should return 300"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

Exercise 9: Inventory Management System

Exercise Description

Create a class `Product` with attributes `name`, `price`, and `quantity`. Add a method `total_value` that calculates the total value of the product (`price * quantity`). Then, create an `Inventory` class that manages multiple `Product` objects. The `Inventory` class should have methods to:

- Add a product.
- Remove a product by name.
- Calculate the total inventory value.

Example:

```
p1 = Product("Laptop", 1500, 10)
p2 = Product("Phone", 800, 20)
inventory = Inventory()
inventory.add_product(p1)
inventory.add_product(p2)
print(inventory.total_value()) # Output: 31000
```

After creating the classes, create the example scenario above and store the inventory in a variable called `result`.

Your solution

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
```

```

        self.quantity = quantity

    def total_value(self):
        return self.price * self.quantity

class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def remove_product(self, name):
        new_products = []
        for product in self.products:
            if product.name != name:
                new_products.append(product)
        self.products = new_products

    def total_value(self):
        total = 0
        for product in self.products:
            total += product.total_value()
        return total

# Create the example scenario
p1 = Product("Laptop", 1500, 10)
p2 = Product("Phone", 800, 20)
result = Inventory()
result.add_product(p1)
result.add_product(p2)

```

Test your solution

```
# Test the Product class
p_test = Product("Test", 100, 5)
assert hasattr(p_test, 'name'), "Product should have a name attribute"
assert hasattr(p_test, 'price'), "Product should have a price attribute"
assert hasattr(p_test, 'quantity'), "Product should have a quantity attribute"
assert hasattr(p_test, 'total_value'), "Product should have a total_value method"
assert p_test.total_value() == 500, "Product total_value should be 500 (100 * 5)"
```

Test your solution

```
# Test the Inventory class
assert hasattr(result, 'products'), "Inventory should have a products attribute"
assert hasattr(result, 'add_product'), "Inventory should have an add_product method"
assert hasattr(result, 'remove_product'), "Inventory should have a remove_product method"
assert hasattr(result, 'total_value'), "Inventory should have a total_value method"
assert len(result.products) == 2, "Inventory should have 2 products"
assert result.total_value() == 31000, "Total inventory value should be 31000 (15000 + 16000)"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Library System

Exercise Description

Create a `Book` class with attributes `title`, `author`, and `is_checked_out` (default to `False`). Add methods `check_out` and `return_book` to update `is_checked_out`. Create a `Library` class that manages a collection of books. The `Library` class should have methods to:

- Add a book.
- Check out a book by title (if it's available).
- Return a book by title.
- List all books and their status.

Example:

```
b1 = Book("1984", "George Orwell")
b2 = Book("To Kill a Mockingbird", "Harper Lee")
library = Library()
library.add_book(b1)
library.add_book(b2)
library.check_out("1984")
```

After creating the classes, create the example scenario above and store the library in a variable called `result`.

Your solution

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_checked_out = False

    def check_out(self):
        if not self.is_checked_out:
            self.is_checked_out = True
            return True
        return False

    def return_book(self):
        self.is_checked_out = False

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def check_out(self, title):
        for book in self.books:
            if book.title == title and not book.is_checked_out:
                book.check_out()
                return f"{title} checked out"
        return "Book not available"
```

```

def return_book(self, title):
    for book in self.books:
        if book.title == title and book.is_checked_out:
            book.return_book()
            return f"{title} returned"
    return "Book not found or not checked out"

def list_books(self):
    for book in self.books:
        status = "Checked Out" if book.is_checked_out else "Available"
        print(f"{book.title} - {status}")

# Create the example scenario
b1 = Book("1984", "George Orwell")
b2 = Book("To Kill a Mockingbird", "Harper Lee")
result = Library()
result.add_book(b1)
result.add_book(b2)
result.check_out("1984")

```

Test your solution

```

# Test the Book class
b_test = Book("Test Book", "Test Author")
assert hasattr(b_test, 'title'), "Book should have a title attribute"
assert hasattr(b_test, 'author'), "Book should have an author attribute"
assert hasattr(b_test, 'is_checked_out'), "Book should have an is_checked_out attribute"
assert b_test.is_checked_out == False, "Book should start as not checked out"
assert hasattr(b_test, 'check_out'), "Book should have a check_out method"
assert hasattr(b_test, 'return_book'), "Book should have a return_book method"

```


Test your solution

```
# Test the Library class and scenario
assert hasattr(result, 'books'), "Library should have a books attribute"
assert len(result.books) == 2, "Library should have 2 books"
assert result.books[0].title == "1984", "First book should be 1984"
assert result.books[0].is_checked_out == True, "1984 should be checked out"
assert result.books[1].is_checked_out == False, "To Kill a Mockingbird should be available"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML; import urllib.parse as u; HTML('<a href="https://pyth
```

Exercise 11: Employee Management System with Inheritance

Exercise Description

Create a base class `Employee` with attributes `name`, `id`, and `salary`. Add a method `get_details` to display basic details. Create two subclasses `Manager` and `Developer` that

inherit from `Employee`.

- `Manager` should have an additional attribute `team_size`.
- `Developer` should have an additional attribute `programming_language`. Each subclass should override the `get_details` method to display specific information.

Example:

```
m = Manager("Alice", 1, 75000, 5)
d = Developer("Bob", 2, 60000, "Python")
print(m.get_details())
print(d.get_details())
# Output: Manager: Alice, ID: 1, Salary: 75000, Team Size: 5
#           Developer: Bob, ID: 2, Salary: 60000, Language: Python
```

After creating the classes, create the example scenario above and store the manager and developer in variables called `result1` and `result2`.

Your solution

```
class Employee:
    def __init__(self, name, id, salary):
        self.name = name
        self.id = id
        self.salary = salary

    def get_details(self):
        return f"Employee: {self.name}, ID: {self.id}, Salary: {self.salary}"
```

```

class Manager(Employee):
    def __init__(self, name, id, salary, team_size):
        super().__init__(name, id, salary)
        self.team_size = team_size

    def get_details(self):
        return f"Manager: {self.name}, ID: {self.id}, Salary: {self.salary}, Team Size: {self.team_size}"

class Developer(Employee):
    def __init__(self, name, id, salary, programming_language):
        super().__init__(name, id, salary)
        self.programming_language = programming_language

    def get_details(self):
        return f"Developer: {self.name}, ID: {self.id}, Salary: {self.salary}, Language: {self.programming_language}"

# Create the example scenario
result1 = Manager("Alice", 1, 75000, 5)
result2 = Developer("Bob", 2, 60000, "Python")

```

Test your solution

```

# Test the Employee base class
e_test = Employee("Test", 999, 50000)
assert hasattr(e_test, 'name'), "Employee should have a name attribute"
assert hasattr(e_test, 'id'), "Employee should have an id attribute"
assert hasattr(e_test, 'salary'), "Employee should have a salary attribute"
assert hasattr(e_test, 'get_details'), "Employee should have a get_details method"

```

Test your solution

```
# Test the Manager class
assert isinstance(result1, Manager), "result1 should be a Manager instance"
assert isinstance(result1, Employee), "result1 should also be an Employee instance (i
assert hasattr(result1, 'team_size'), "Manager should have a team_size attribute"
assert result1.get_details() == "Manager: Alice, ID: 1, Salary: 75000, Team Size: 5"

# Test the Developer class
assert isinstance(result2, Developer), "result2 should be a Developer instance"
assert isinstance(result2, Employee), "result2 should also be an Employee instance (i
assert hasattr(result2, 'programming_language'), "Developer should have a programming
assert result2.get_details() == "Developer: Bob, ID: 2, Salary: 60000, Language: Pyth
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: E-commerce Order System

Exercise Description

Create a `Product` class with attributes `name` and `price`. Create an `Order` class that manages a list of `Product` objects. The `Order` class should have methods to:

- Add a product to the order.
- Remove a product by name.
- Calculate the total order value.

Example:

```
p1 = Product("Laptop", 1000)
p2 = Product("Headphones", 50)
order = Order()
order.add_product(p1)
order.add_product(p2)
print(order.total_order_value()) # Output: 1050
```

After creating the classes, create the example scenario above and store the order in a variable called `result`.

Your solution

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class Order:
    def __init__(self):
        self.products = []

    def add_product(self, product):
```

```

        self.products.append(product)

    def remove_product(self, name):
        new_products = []
        for product in self.products:
            if product.name != name:
                new_products.append(product)
        self.products = new_products

    def total_order_value(self):
        total = 0
        for product in self.products:
            total += product.price
        return total

# Create the example scenario
p1 = Product("Laptop", 1000)
p2 = Product("Headphones", 50)
result = Order()
result.add_product(p1)
result.add_product(p2)

```

Test your solution

```

# Test the Product class
p_test = Product("Test", 100)
assert hasattr(p_test, 'name'), "Product should have a name attribute"
assert hasattr(p_test, 'price'), "Product should have a price attribute"
assert p_test.name == "Test", "Product name should be Test"
assert p_test.price == 100, "Product price should be 100"

```

Test your solution

```
# Test the Order class
assert hasattr(result, 'products'), "Order should have a products attribute"
assert hasattr(result, 'add_product'), "Order should have an add_product method"
assert hasattr(result, 'remove_product'), "Order should have a remove_product method"
assert hasattr(result, 'total_order_value'), "Order should have a total_order_value method"
assert len(result.products) == 2, "Order should have 2 products"
assert result.total_order_value() == 1050, "Total order value should be 1050 (1000 + 50)"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: School Grading System

Exercise Description

Create a `Student` class with attributes `name`, `grades` (a list of integers), and methods:

- `add_grade` to add a grade.
- `average_grade` to calculate the average grade. Create a `Course` class that manages a list of `Student` objects. The `Course` class should have methods to:
 - Add a student.
 - Display all students' names and their average grade.

Example:

```
s1 = Student("Alice", [90, 80, 85])
s2 = Student("Bob", [70, 75, 80])
course = Course()
course.add_student(s1)
course.add_student(s2)
course.display_students()
# Output: Alice - Average Grade: 85.0
#         Bob - Average Grade: 75.0
```

After creating the classes, create the example scenario above and store the course in a variable called `result`.

Your solution

```
class Student:
    def __init__(self, name, grades=None):
        self.name = name
        self.grades = grades if grades else []

    def add_grade(self, grade):
```



```

        self.grades.append(grade)

    def average_grade(self):
        return sum(self.grades) / len(self.grades) if self.grades else 0

class Course:
    def __init__(self):
        self.students = []

    def add_student(self, student):
        self.students.append(student)

    def display_students(self):
        for student in self.students:
            print(f"{student.name} - Average Grade: {student.average_grade():.2f}")

# Create the example scenario
s1 = Student("Alice", [90, 80, 85])
s2 = Student("Bob", [70, 75, 80])
result = Course()
result.add_student(s1)
result.add_student(s2)

```

Test your solution

```

# Test the Student class
s_test = Student("Test", [80, 90, 100])
assert hasattr(s_test, 'name'), "Student should have a name attribute"
assert hasattr(s_test, 'grades'), "Student should have a grades attribute"
assert hasattr(s_test, 'add_grade'), "Student should have an add_grade method"

```

